Signal Object Detection

Machine Learning Project Report

Ștefan Chiper

1 Data Processing

1.1 Dataset and Initial Observations

The objective of the project is to classify signal images into 5 classes, where each class corresponds to the number of visible signals/objects in the image. Consequently, the problem combines standard multi-class image classification with an implicit counting component.

Prior to model training, I performed a qualitative inspection of representative samples from each class in order to characterize the visual structure of the dataset and identify the discriminative patterns that the model must learn. The image below shows representative examples for classes 1–5.

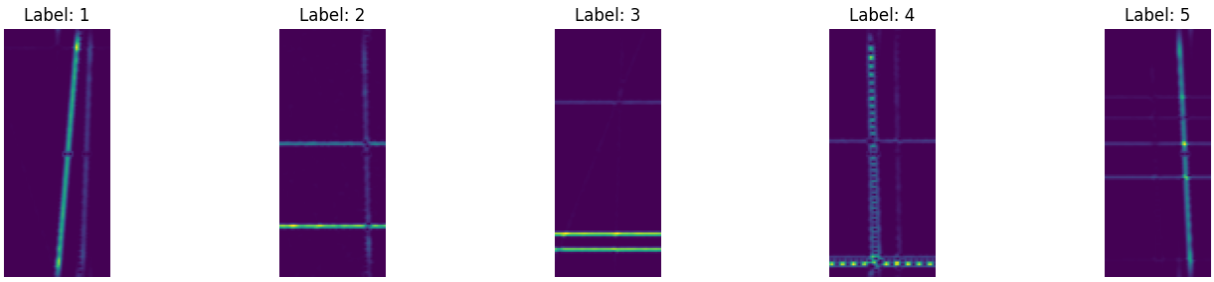

Figure 1: Examples of images from the dataset for classes 1–5.

Visual inspection of the dataset revealed the following characteristics:

- the background is predominantly dark, while the signals are represented by high-intensity regions;
- signal structures may be vertical, horizontal, or composed of both orientations;
- signal localization varies across samples;
- adjacent classes can exhibit substantial visual similarity, particularly when signals have low intensity or partially overlap;
- image noise can reduce class separability and increase classification difficulty;

- preserving the spatial arrangement of signal components is important for accurate classification.

Based on these observations, KNN was used only as a lightweight baseline, while the primary solution was implemented using a convolutional neural network capable of learning local spatial features.

1.2 Preprocessing for KNN

For the KNN baseline, I used a simplified image representation. Each image is loaded, converted to grayscale, resized to 64×64, scaled to the $[0, 1]$ range by division by 255, and flattened into a one-dimensional feature vector.

The corresponding preprocessing code is shown below:

```
img = Image.open(folder + "/" + str(file_name)).convert("L")
img = img.resize(img_size)
hist = np.asarray(img, dtype=np.float32) / 255.0
hist = hist.reshape(-1)
```

This representation is computationally inexpensive and convenient for rapid experimentation, but it has an important limitation: flattening removes the explicit two-dimensional spatial relationships between neighboring pixels.

1.3 Preprocessing for CNN

For the CNN pipeline, the images were preserved in RGB format and resized to 256×110. The training-set mean and standard deviation were computed and subsequently used for input normalization.

This normalization standardizes the input distribution and improves numerical stability during optimization. The validation and test pipelines contain no stochastic augmentations; they apply only resizing, tensor conversion, and normalization.

1.4 Augmentations

Data augmentation was performed online during training. The augmentation policy applies at most one transformation per image in order to increase sample diversity without excessively distorting the underlying signal structure.

Augmentation	Probability	Reason
Mosaic4	0.10	combines four training samples
Mosaic2	0.12	combines two training samples
SpecAugment	0.20	improves robustness to masking and noise
Horizontal Flip	0.18	improves invariance to horizontal mirroring
Affine	0.20	applies small translations and scale changes

Table 1: Augmentations used during training.

Mosaic augmentation is particularly relevant because the target classes encode signal counts. When multiple images are combined, the resulting target is defined as the sum of the component labels, provided that the resulting class does not exceed 5.

SpecAugment was included because the input images resemble spectrogram-like representations whose axes may be interpreted in terms of time and frequency. Masking horizontal or vertical bands discourages the network from relying on isolated local regions and promotes learning of more distributed signal structure and contextual information.

Affine perturbations were intentionally constrained to small magnitudes because aggressive geometric transformations could alter the semantic structure of the signal.

2 Models Tried

2.1 KNN - Baseline

KNN was used as the initial baseline model. Its primary purpose was to validate the end-to-end data-loading and inference pipeline while providing a simple reference point for subsequent CNN experiments.

The baseline was evaluated across multiple hyperparameter configurations:

- metrics: `euclidean`, `cosine`;
- weights: `uniform`, `distance`;
- different values for the number of neighbors k.

The values tested for k were:

```
[1, 3, 5, 7, 9, 11, 15, 21, 31, 41, 61, 81, 101]
```

For each configuration, validation accuracy was measured, after which the best-performing configuration was retrained on the complete training set.

Although KNN is simple and efficient for baseline experiments, its performance was limited because distance-based classification on flattened vectors does not explicitly exploit local spatial structure.

2.2 SignalCNN - Main Model

The primary model, SignalCNN, is a convolutional neural network trained from scratch. This architecture was selected because convolutional operators provide an appropriate inductive bias for extracting local spatial patterns from image data.

The network architecture consists of:

- an initial convolutional stem followed by Batch Normalization and ReLU activation;
- residual feature-extraction blocks;
- Squeeze-and-Excitation channel-attention modules after the residual blocks;
- Global Average Pooling for spatial aggregation;
- a multi-class classification head;
- an auxiliary regression head.

The high-level structure of the convolutional backbone is summarized below:

Stage	Channels	Observation
Stem	32	low-level feature extraction
Level 1	64	residual feature extraction + SE
Level 2	128	increased representational capacity
Level 3	256	higher-level feature extraction
Level 4	512	high-level latent representation

Table 2: Layer-wise structure of the CNN model.

Residual blocks facilitate optimization of a deeper architecture by introducing shortcut connections that improve information and gradient propagation. Squeeze-and-Excitation modules perform adaptive channel-wise recalibration, allowing the network to emphasize more informative feature channels and suppress less relevant responses.

2.3 The Two Model Heads

SignalCNN uses a dual-head output architecture:

- a five-class classification output;
- an auxiliary regression output defined over a normalized target range.

The classification head produces the final categorical prediction. The auxiliary regression head is used only as an additional training objective and does not directly determine the

predicted class. Its purpose is to exploit the ordinal structure of the labels: for example, class 1 is semantically closer to class 2 than to class 5.

This auxiliary objective is therefore consistent with the counting-related structure of the task.

3 Training and Validation

3.1 Configuration Used

The principal hyperparameter configuration used for the final model was:

```
NUM_CLASSES = 5
IMAGE_HEIGHT = 256
IMAGE_WIDTH = 110
BATCH_SIZE = 16
N_SPLITS = 5
SEED = 42

ALPHA = 0.50
EPOCHS_STAGE1 = 90
EPOCHS_STAGE2 = 90
EARLY_STOP_PATIENCE = 35

LEARNING_RATE = 1e-4
WEIGHT_DECAY = 5e-4
LABEL_SMOOTHING = 0.02

PSEUDO_LABEL_THRESHOLD = 0.90
USE_TTA_HFLIP = True
```

Optimization was performed with AdamW. The classification branch was trained using CrossEntropy loss, while the auxiliary regression branch used SmoothL1 loss. The total training objective is a weighted combination of these two losses, with `ALPHA` controlling the contribution of the regression component.

Label smoothing was applied to reduce overconfident class probabilities and improve regularization, which is particularly relevant when high-confidence predictions are subsequently used for pseudo-labeling.

3.2 K-Fold Cross Validation

To obtain a more reliable estimate of generalization performance, I used 5-fold Stratified K-Fold cross-validation. For each fold, the model is trained on four partitions and evaluated on the remaining held-out partition while preserving the class distribution.

This procedure produces out-of-fold predictions for every training example using a model that did not observe that example during optimization. At inference time, predictions from

the five fold-specific models are averaged, forming an ensemble that reduces dependence on any single train–validation split.

3.3 Early Stopping

For each fold, the checkpoint achieving the highest validation accuracy is retained. Training is terminated automatically when the validation metric fails to improve for the configured patience interval.

Early stopping avoids unnecessary optimization after convergence and reduces the risk of overfitting to the fold-specific training subset.

3.4 Test-Time Augmentation

During inference, horizontal-flip Test-Time Augmentation (TTA) was applied. The final class-probability vector is obtained by averaging predictions from the original image and its horizontally mirrored counterpart.

This lightweight inference-time ensemble can improve prediction robustness when signal structures occur at slightly different spatial locations.

3.5 Pseudo-Labeling

After Stage 1, the fold ensemble generates predictions for the test set. Samples whose maximum predicted class probability is at least 0.90 are selected as high-confidence pseudo-labeled examples and added to the training data with their predicted labels.

Stage 2 is then trained using the original labeled data together with the selected pseudo-labeled samples. Validation remains restricted to the original labeled data, preventing pseudo-labeled examples from artificially inflating the evaluation metric.

3.6 Blend Between Stages

The final prediction pipeline blends the probability outputs from Stage 1 and Stage 2. The interpolation weight is selected using out-of-fold predictions, and the weight yielding the highest OOF accuracy is used to construct the final submission.

This probability-level ensemble is beneficial because the two training stages exhibit partially complementary error patterns; combining their outputs can therefore produce more robust predictions.

4 Results

4.1 KNN Results

KNN provided a useful sanity-check baseline but was insufficient for competitive performance. Flattening each image into a feature vector forces the classifier to operate on global pairwise distances and removes the explicit spatial organization of the signal patterns.

Nevertheless, the KNN baseline was useful for:

- validating the data-loading pipeline;
- rapidly verifying the end-to-end submission pipeline;
- establishing a reference point for comparison with the CNN model.

For the KNN experiments, I evaluated validation accuracy as a function of the neighborhood size and distance/weighting configuration. The following plot compares the tested variants and supports selection of the strongest baseline configuration.

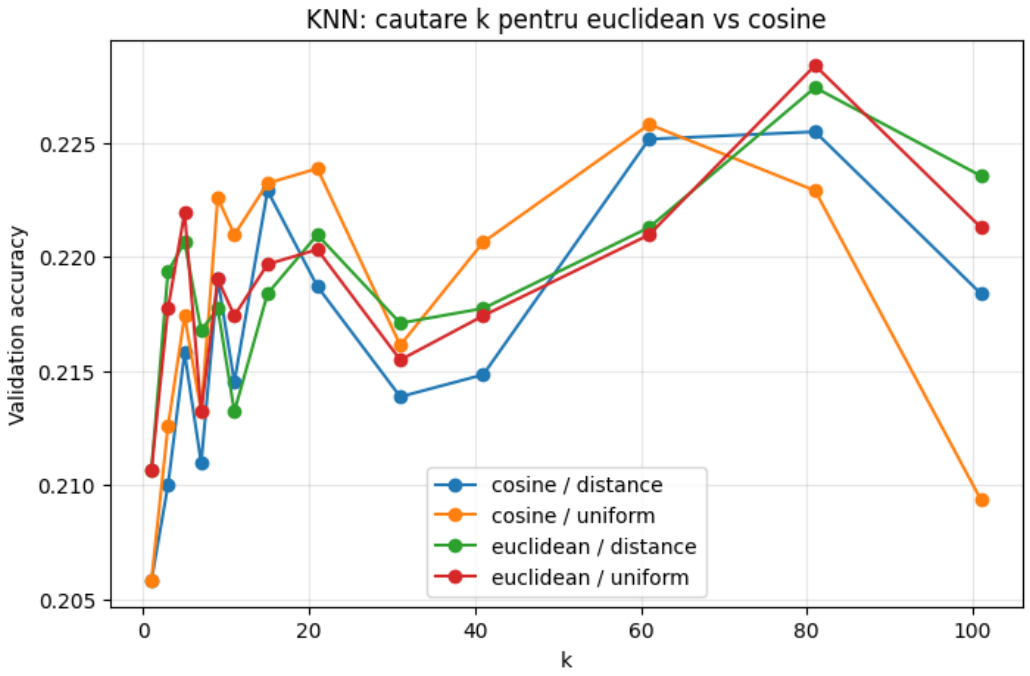

Figure 2: Comparison between the tested KNN configurations.

After selecting the best KNN configuration, I analyzed its confusion matrix to characterize class-specific error patterns. The matrix confirms that the distance-based baseline has difficulty separating classes with similar visual structure.

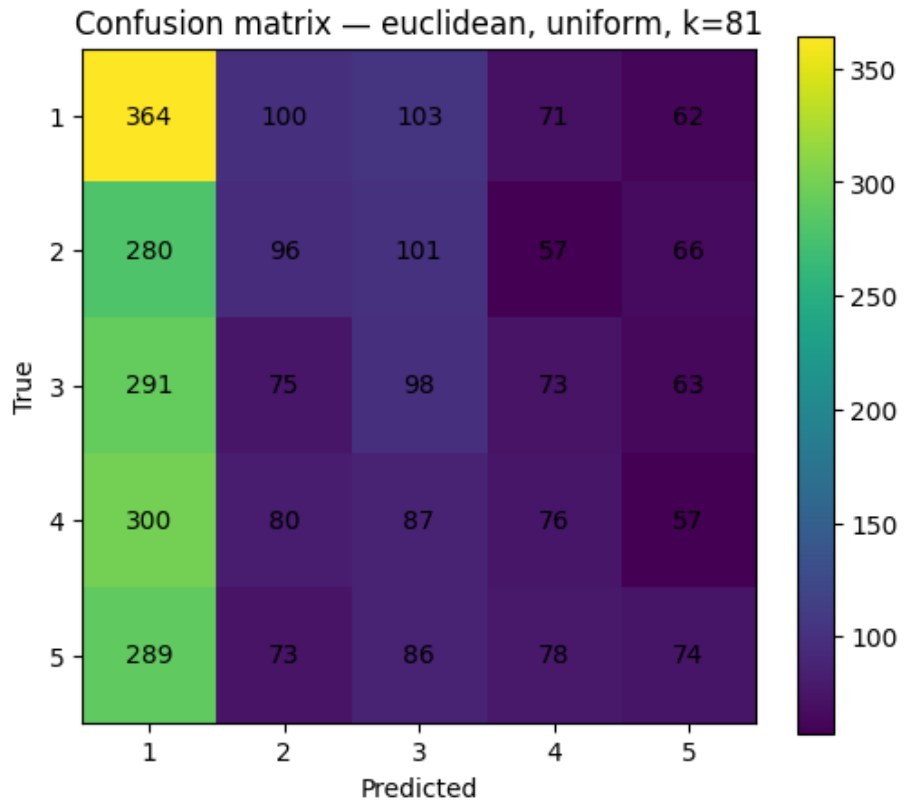

Figure 3: Confusion matrix for the best KNN configuration.

Although several KNN configurations provide a reasonable baseline, the method remains fundamentally constrained by its flattened representation and lack of explicit spatial feature extraction. This motivated the transition to a convolutional architecture.

4.2 CNN Results

The CNN-based approach achieved substantially stronger performance. The main components contributing to the final system were:

- retaining the full RGB input representation;
- using the higher input resolution;
- residual feature-extraction blocks;
- Squeeze-and-Excitation channel recalibration;
- the auxiliary ordinal regression objective;
- stratified K-Fold cross-validation and fold ensembling;
- confidence-based pseudo-labeling;
- Test-Time Augmentation and probability-level stage blending.

For Stage 1, validation accuracy was monitored throughout training for each fold. The learning curves indicate stable optimization across folds, with performance continuing to improve toward the later epochs.

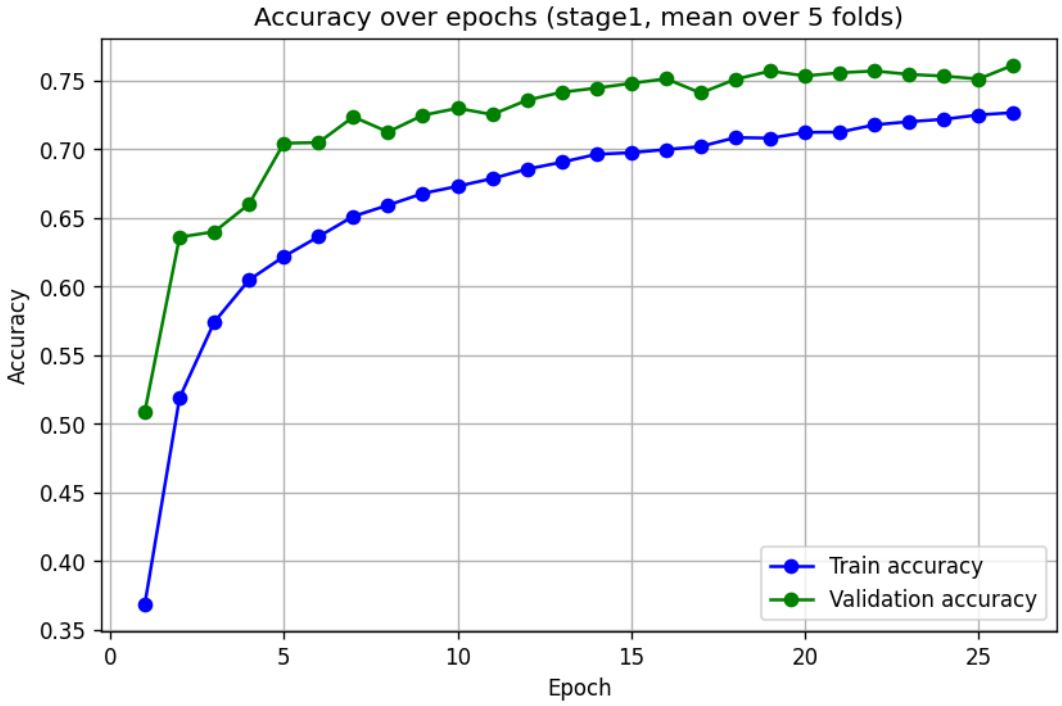

Figure 4: Evolution of accuracy in Stage 1.

The Stage 1 loss curves were also inspected to assess convergence behavior and optimization stability. The observed trajectories are consistent with stable training and progressive convergence.

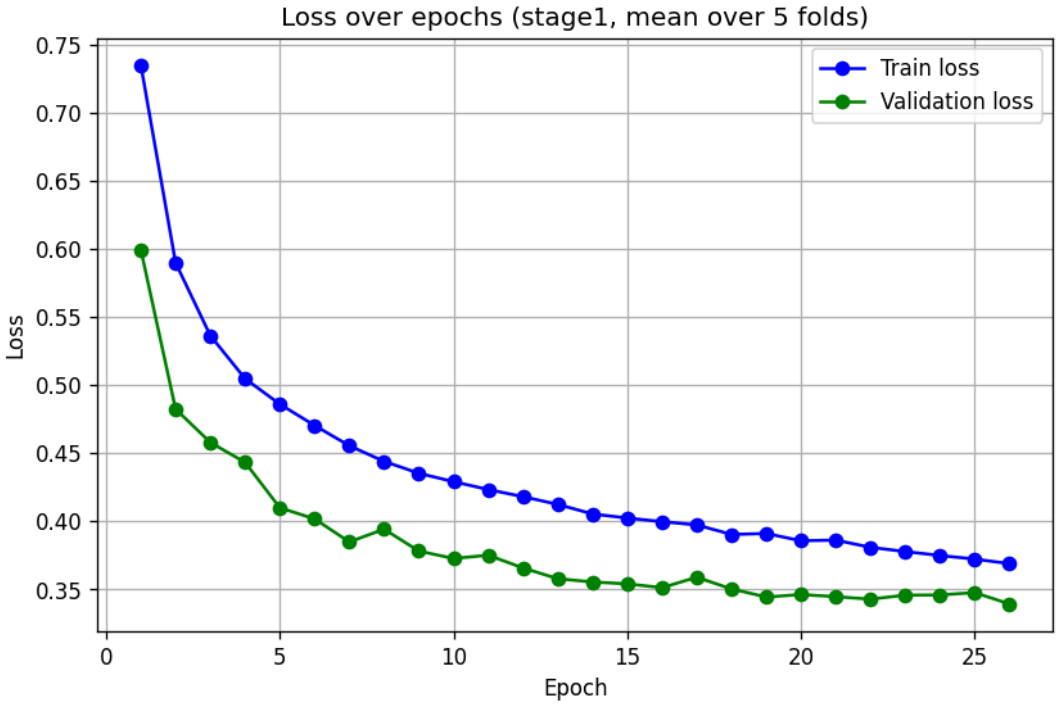

Figure 5: Evolution of the loss in Stage 1.

The Stage 1 confusion matrix highlights the dominant class-level error modes. The model exhibits a tendency toward underestimation, with the most frequent misclassifications occurring between a target class and the immediately preceding class.

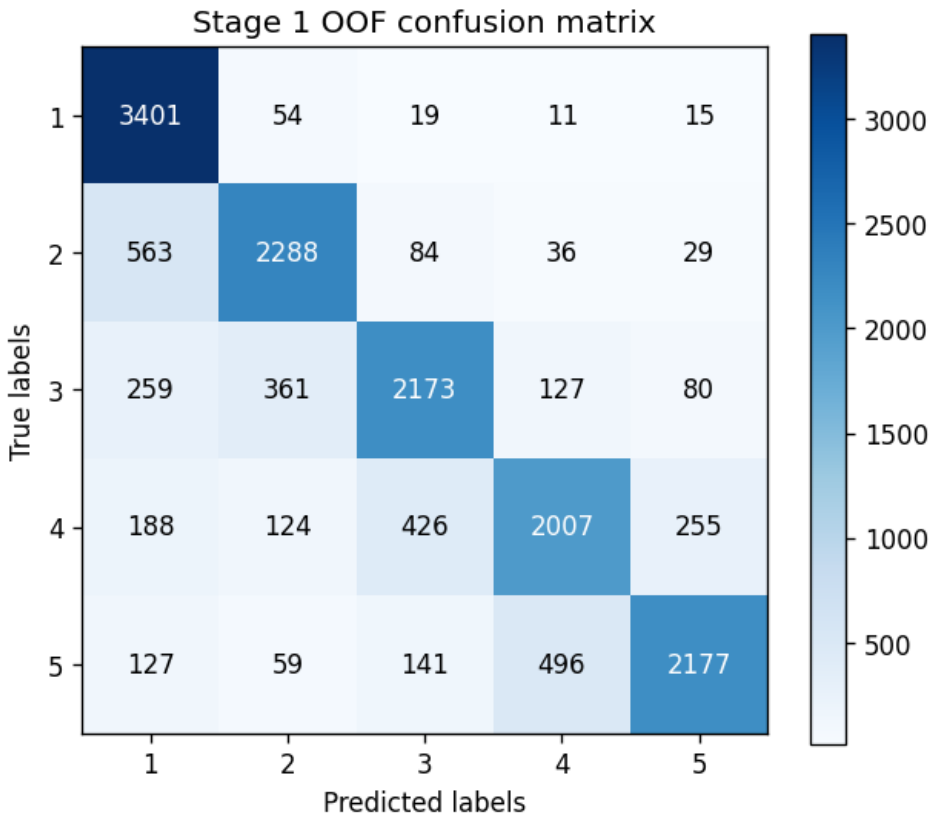

Figure 6: Confusion matrix for Stage 1.

Following pseudo-label generation, Stage 2 was trained and its validation accuracy was monitored using the same procedure. The resulting curves indicate an improvement relative to Stage 1.

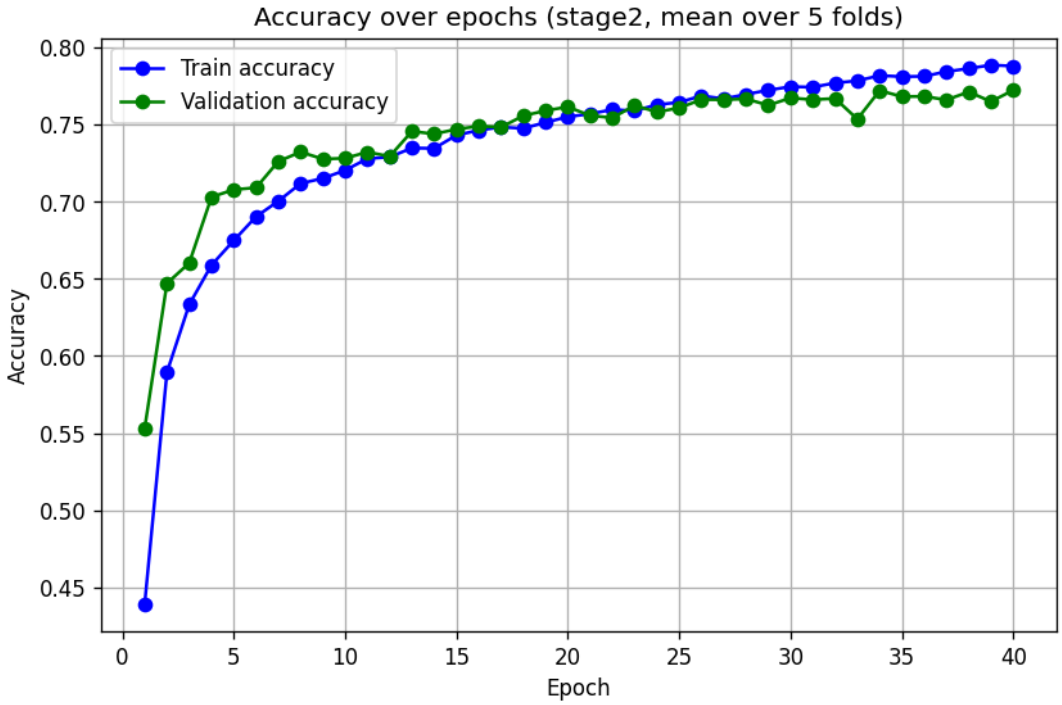

Figure 7: Evolution of accuracy in Stage 2.

The Stage 2 loss curves were examined to verify that introducing pseudo-labeled samples did not destabilize optimization. Their behavior remains well-formed, suggesting that the selected pseudo-labels did not introduce excessive training noise.

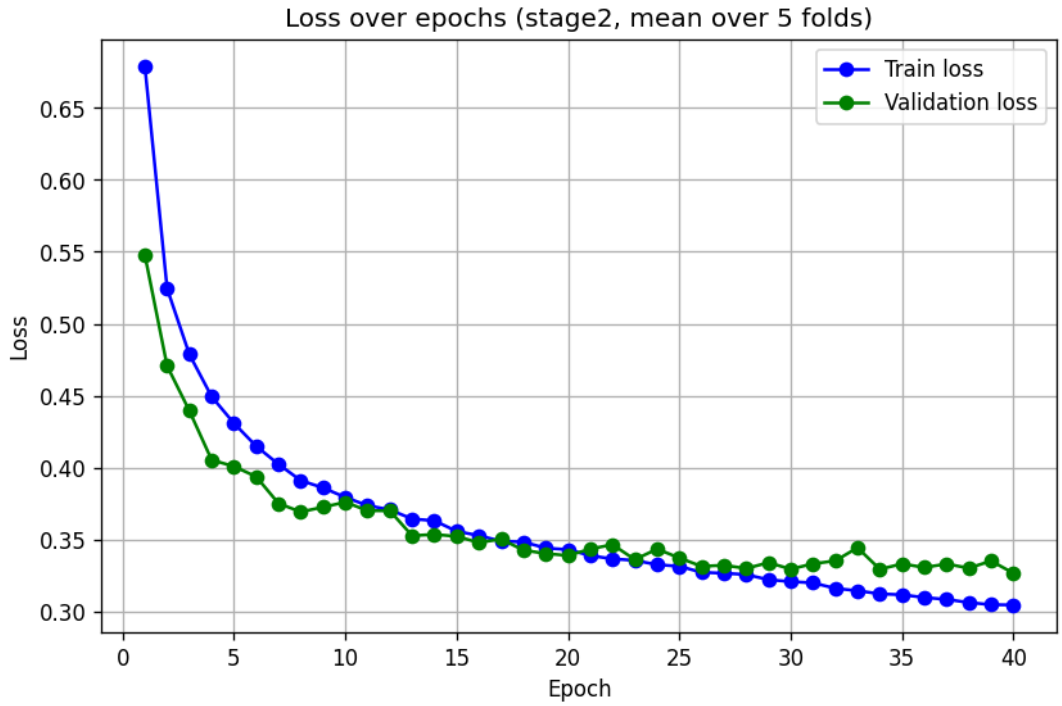

Figure 8: Evolution of the loss in Stage 2.

The Stage 2 confusion matrix illustrates how the class-specific error distribution changes

after pseudo-labeling. Performance improves for classes 4 and 5, while classes 1 and 2 exhibit a slight decrease.

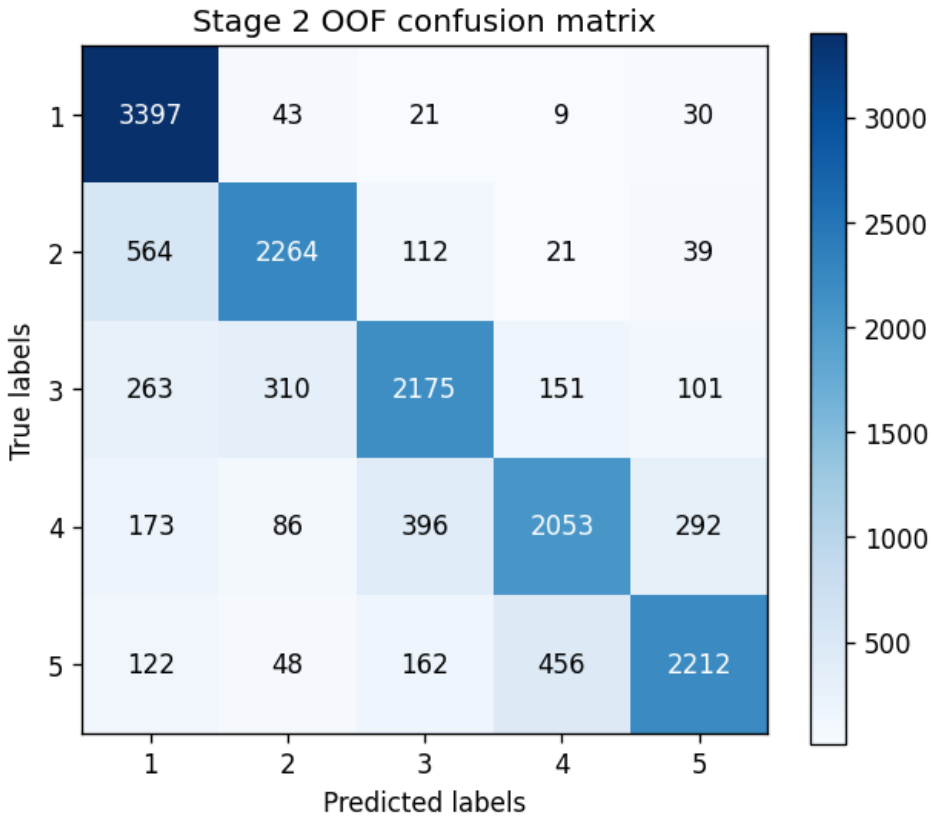

Figure 9: Confusion matrix for Stage 2.

After obtaining predictions from both stages, I performed a search over the blend coefficient used to interpolate their probability outputs. The following plot shows OOF accuracy as a function of the blending weight.

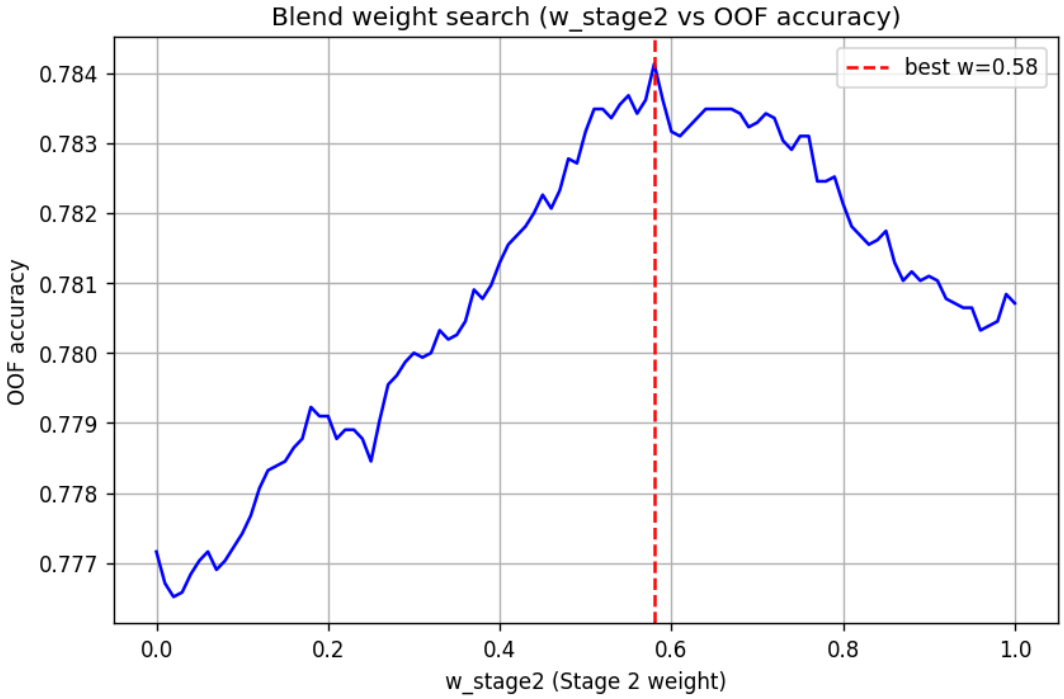

Figure 10: Search for the optimal blend weight.

For the selected blend, I also analyzed the out-of-fold confusion matrix. This provides the most representative comparison of the final ensemble because it is computed from the same blended OOF predictions used to select the final combination strategy.

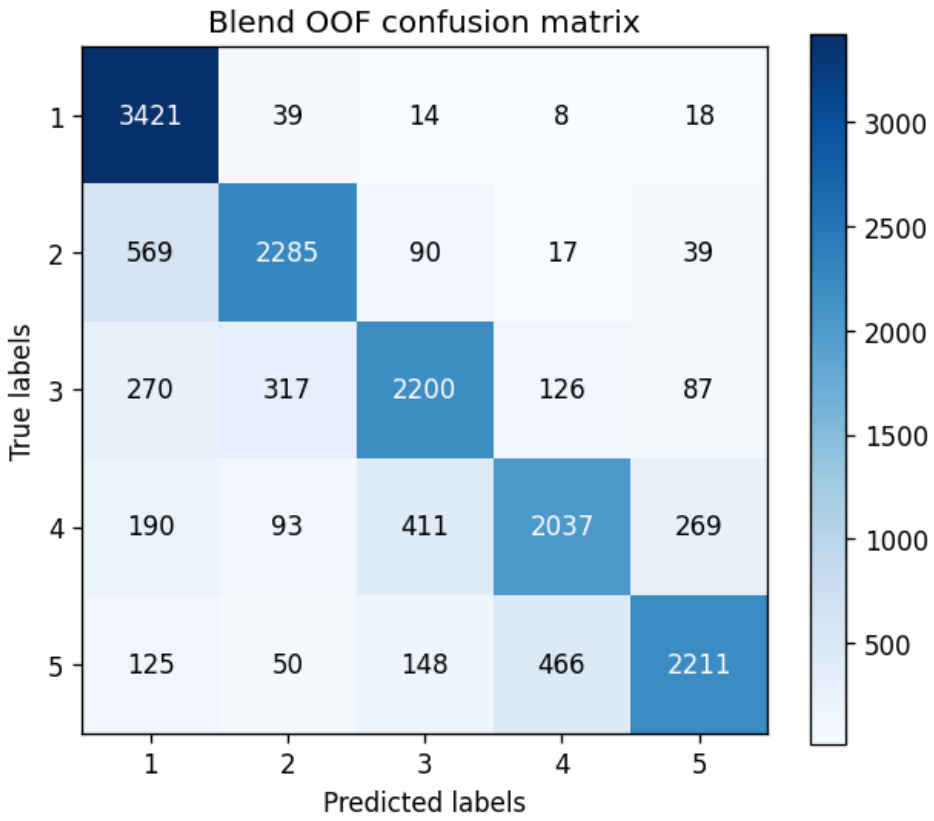

Figure 11: Confusion matrix for the final OOF blend.

I subsequently compared per-class accuracy across the principal model variants to determine whether the improvement was class-specific or broadly distributed. The blended model achieves the highest accuracy for every class.

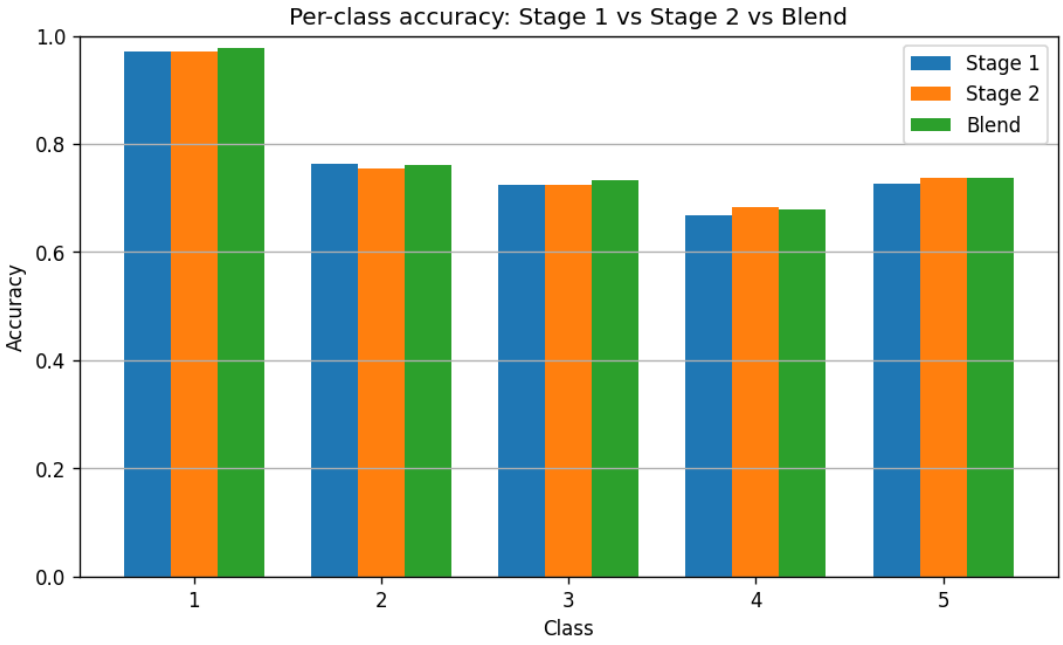

Figure 12: Comparison of per-class accuracies.

Finally, I inspected the predicted class distribution on the test set. The distribution reveals a pronounced preference for class 1, whose predicted sample count is almost twice that of any other individual class.

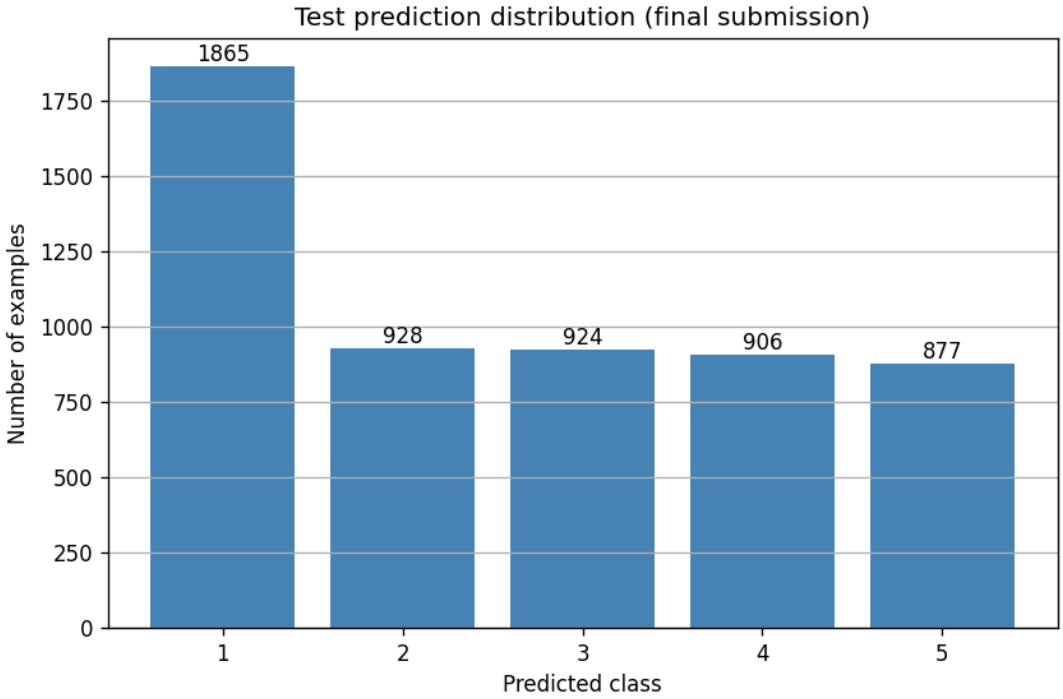

Figure 13: Distribution of predictions on the test set.

The strongest configuration was SignalCNN with Squeeze-and-Excitation modules after all residual blocks, a dual-head classification/regression objective, and pseudo-labeling with a confidence threshold of 0.90. The best score obtained on the final leaderboard was approximately:

$$0.80436$$

Compared with KNN, the CNN can explicitly learn hierarchical local features and preserve spatial relationships between signal components. Furthermore, K-Fold ensembling and the final stage blend reduce variance relative to relying on a single model trained on one fixed train–validation split.

5 Conclusions

The experimental pipeline progressed from a simple KNN baseline to a custom CNN architecture tailored to the spatial characteristics of the signal images. The substantial performance gain was obtained from the convolutional approach and the associated training and ensembling strategy.

The main technical conclusions are:

- preserving spatial structure is essential for this type of image data;
- KNN is useful as a lightweight baseline but is limited by its flattened feature representation;
- convolutional feature extraction provides a substantially better representation of signal morphology and spatial localization;
- Squeeze-and-Excitation improves channel-wise feature recalibration;
- the auxiliary regression objective exploits the natural ordinal structure of the classes;
- K-Fold ensembling and confidence-based pseudo-labeling improve prediction robustness;
- probability-level blending of Stage 1 and Stage 2 improves performance relative to using either stage independently.

References

- K. He, X. Zhang, S. Ren, J. Sun, *Deep Residual Learning for Image Recognition*, CVPR 2016.
  https://arxiv.org/abs/1512.03385
- J. Hu, L. Shen, S. Albanie, G. Sun, E. Wu, *Squeeze-and-Excitation Networks*, CVPR 2018.
  https://arxiv.org/abs/1709.01507
- D. S. Park, W. Chan, Y. Zhang, C.-C. Chiu, B. Zoph, E. D. Cubuk, Q. V. Le, *SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition*, Interspeech 2019.
  https://arxiv.org/abs/1904.08779
- PyTorch Documentation, `Dataset`, `DataLoader`, `CrossEntropyLoss`, `SmoothL1Loss`, `AdamW`.
  https://pytorch.org/docs/stable/index.html